Retrieval-Augmented Generation

Part 1: finding the right context

A question about your own factory

The maintenance engineer asks a strong language model:

> *"What pressure should the Lori cheese press run at?"*

It answers immediately, fluently, and in full sentences:

> *"Typically around 4 to 6 bar, depending on the mould size."*

It has never seen your factory.

The number is invented. Nothing in the answer tells you that.

The same question, with one paragraph attached

Context: "The Lori press operates at 2.5 bar during the first pressing stage. After twenty minutes the pressure is raised to 3.2 bar for the second stage."

Question: What pressure should the Lori cheese press run at?

"2.5 bar for the first stage, raised to 3.2 bar after twenty minutes." [source: press manual, rev. 4]

Same model. Same question. The only thing that changed is what was in front of it.

That is the whole idea

Retrieval-Augmented Generation (RAG) means: before answering, go and fetch the relevant text, and put it in the prompt.

The idea takes one slide. The engineering takes the rest of the chapter, because of one question:

How do you find **that one paragraph**, out of **4,000 documents**,
in **200 milliseconds**, for a question phrased in words
that may appear **nowhere in the document**?

Everything in this lecture is about that search. The generation half is the easy half.

Outline

Why retrieve at all

Chunking

Finding chunks by words

Finding chunks by meaning

When your language is not English

Why retrieve at all

The model already knows a lot. It just does not know anything about you.

What a trained model actually knows

Everything the model knows is frozen into its weights at training time. That gives you four problems at once:

It is out of date.
Training stopped on some date. Yesterday's price change is not in there.

It cannot cite.
A fact recalled from weights has no source attached to it.

It never saw your data.
Your manuals, contracts and incident reports were never public.

It has no permissions.
A model cannot show one answer to a manager and another to an intern.

Retrieval fixes all four at once, because the answer arrives as **text you control**, not as a memory you hope is right.

The obvious objection

"Modern models have huge context windows. Why not paste all 4,000 documents?"

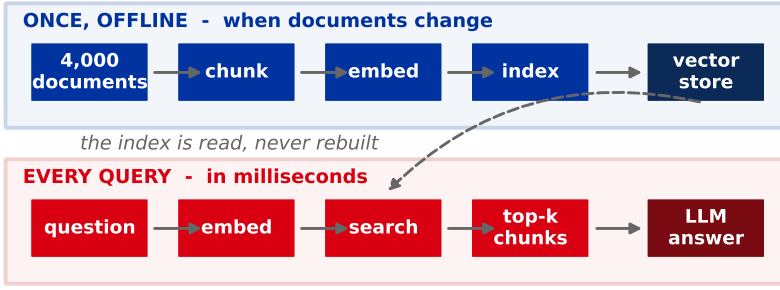
Three reasons, in increasing order of how much they hurt:

1. **It does not fit.** A few thousand documents is tens of millions of tokens. No window is that large.
2. **It is expensive.** You pay per token, on *every single question*, for 4,000 documents of which one is relevant.
3. **It gets worse answers.** Accuracy drops when the relevant sentence is buried in a long context, rather than sitting at the top of a short one.

Retrieval is not a workaround for small context windows. Giving the model **less, but the right less**, is what makes the answer good.

RAG is two pipelines, not one

RAG is two pipelines, and only one of them runs when a user asks



The expensive work happens **once, offline**. When a user asks something, only the bottom row runs. Hold on to this split: it is the reason half the design decisions later in this lecture make sense.

Chunking

The least glamorous step in the pipeline, and the one that quietly decides your ceiling.

Why not just index whole documents?

Because retrieval returns whatever unit you indexed.

Index whole manuals

You get back a 90-page manual. You have moved the problem, not solved it: the model still has to find the paragraph.

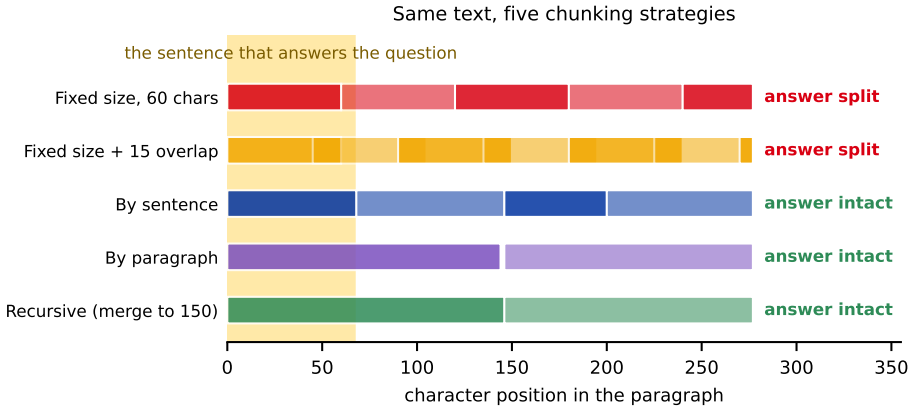
Index single sentences

You get back "It must then be raised." Raised from what? To what? The sentence lost its context.

A **chunk** is the unit you retrieve. It should be small enough to be specific, and large enough to stand alone when it lands in the prompt.

There is a second reason, which returns in Section 4: a chunk becomes *one vector*, and one vector can only represent one meaning.

The five strategies, on the same text



Boundaries computed by running each strategy, not drawn by hand. Note that the two *structural* strategies, sentence and paragraph, both keep the answer intact, while the two *arithmetic* ones destroy it.

Cut where the author already cut

Fixed-size chunking treats text as a character array. But your documents already contain the author's own decisions about where one idea ends:

Boundary	**What it usually means**
sentence break	one statement
paragraph break	one idea, which is usually what you want
heading	one topic
list item	one step of a procedure
table row	one record, and never split one of these

Paragraph chunking is the cheapest good default. It costs one `split` call, needs no model, and respects a boundary a human already placed deliberately. Start here, and only get cleverer if it measurably fails.

The overlap trap

The standard fix for "fixed-size chunking cut my sentence in half" is to make chunks overlap. Look at what that actually measured:

Strategy	Chunks	Answer sentence
Fixed size, 60 characters	5	**split**
Fixed size, 60 characters + 15 overlap	7	**still split**
By sentence	4	**intact**
By paragraph	2	**intact**
Recursive (merge up to 150)	2	**intact**

Overlap did not save it. The answer sentence is 67 characters and the chunks are 60. No overlap puts a 67-character sentence inside a 60-character box. Overlap protects boundaries; it cannot protect a unit of meaning bigger than the chunk.

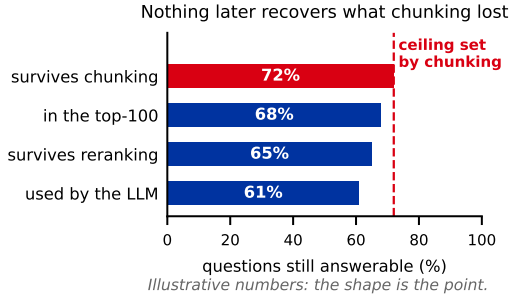
So which one should you use?

Your documents	Start with	Why
Clean prose, real paragraphs	**paragraph**	the author already did the work
Manuals, contracts, headings	**recursive**	respects structure, caps the size
Transcripts, chat logs	**fixed + overlap**	there is no structure to respect
Tables and records	**one row per chunk**	a split row is a wrong answer
Long undivided walls of text	**semantic**	nothing else has a boundary to use

That last row needs a tool we do not have yet. We come back to it at the end of the lecture, once the machinery exists.

There is no best chunking strategy, only a best strategy **for the shape of your documents**. Pick by looking at the documents, then measure, then change your mind.

The recall ceiling



If chunking destroyed the answer, **nothing downstream recovers it**. Not a better embedding model, not a reranker, not a bigger language model. When a RAG system is bad, people tune the model. The problem is usually here.

(*Reranking*, on the axis above, means re-scoring a shortlist with a slower, more accurate model. It gets its own lecture next time.)

Store more than the text

Each chunk should carry the facts you will need *about* it:

Field	**Why you will want it**
source document	so the answer can cite something
section heading	so a chunk that says "it" still has a subject
revision date	so a 2019 procedure loses to the 2024 one
access level	so an intern does not retrieve the salary table

Two of these pay off later: **revision date** decides a contradiction in the next lecture's reranking section, and **access level** is how the permission problem from Section 1 actually gets solved.

The problem that sets up the rest of the lecture

Your chunk says:

> "Ripening cellars are held at 10 degrees and 85 percent humidity."

Your user asks:

> "How warm is the room where the cheese matures?"

The chunk **is** the answer.
They share **no content word at all**.

Remember this pair. We will run it through both retrieval methods.

Finding chunks by words

The method from 1990s search engines that is still, in 2026, half of every serious system.

Start with the dumbest possible thing

Query: `lori press bar`. Count how many query words appear in each chunk.

	Chunk	Query-word occurrences
D1	"The Lori press operates at 2.5 bar…"	3
D2	"Press maintenance… press… press… bar"	4
D3	"Cheese brining… for Lori cheese."	1

D2 wins. But D2 is a maintenance schedule that never mentions Lori cheese, and D1 is the chunk that actually answers the question.

Counting fails for three separate reasons. We will fix them one at a time, and the result will be BM25.

Fix 1: rare words carry more information

"The" appears in nearly every chunk. "Lori" appears in a handful. Matching on "Lori" tells you far more.

$$\text{idf}(t) \;=\; \log \frac{N - n_t + 0.5}{n_t + 0.5}$$

where N is the number of chunks in the corpus and n_t how many of them contain term t. This is the **inverse document frequency**: rare term, big weight.

For our factory archive ($N = 4{,}000$):

Term	Appears in	idf
`lori`	120 chunks	**3.47**
`bar`	210 chunks	2.89
`press`	380 chunks	2.25

This already has a name: tf-idf

Multiply how often a term appears here by how rare it is overall, and sum over the query:

$$\text{tf-idf}(q, d) = \sum_{t \in q} \underbrace{tf_{t,d}}_{\substack{\text{term frequency:} \\ \text{how often, here}}} \cdot \underbrace{\text{idf}(t)}_{\substack{\text{inverse document frequency:} \\ \text{how rare, overall}}}$$

This is the classic weighting behind every search engine built before the 2000s, and it is still the first thing to try. It fixes the "the" problem completely.

A term is worth something when it is frequent here and rare everywhere else. That single sentence is the whole idea, and it survives into BM25 unchanged.

But tf-idf still gets our example wrong

Query `lori press bar`, scored with tf-idf:

	Chunk	tf-idf	
D1	the one that answers the question	8.62	
D2	the maintenance schedule	**9.65**	**wins**
D3	brining	3.47	

D2 wins because it says `press` **three times**, and tf-idf takes that literally: three times the term, three times the weight. It also never notices that D2 is more than twice as long as D1.

So tf-idf fixed *rarity* and nothing else. Two problems remain: repetition counts too much, and long chunks get a free advantage. **BM25 is tf-idf with exactly those two repairs.**

Fix 2: predict first

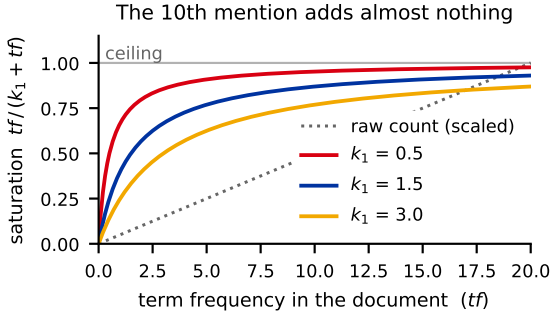
A chunk mentions `pressure` **50 times**.

Another mentions it **once**.

Is the first one 50 times more relevant?

Decide before the next slide.

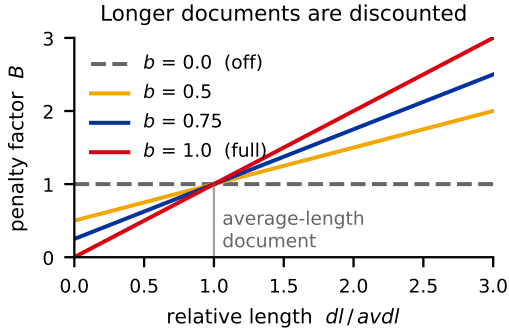
Fix 2: term frequency saturates



$$\text{saturation}(tf) = \frac{tf}{k_1 + tf}$$

The 2nd mention is strong evidence. The 50th is almost none. k_1 sets how fast the curve flattens.

Fix 3: long chunks cheat



A long chunk contains more of every word, so it wins by length, not relevance.

$B = (1 - b) + b\, dl/avdl$ divides that out: dl is the chunk's length, $avdl$ the average.

$b = 0$ turns the correction off, $b = 1$ applies it fully. Usually 0.5 to 0.8.

Putting the three fixes together

$$\text{BM25}(q,d) = \sum_{t \in q} \underbrace{\frac{tf_{t,d}}{k_1\left((1-b)+b\frac{dl}{avdl}\right)+tf_{t,d}}}_{\text{saturates, and discounts long chunks}} \cdot \underbrace{\text{idf}(t)}_{\text{rare terms count more}}$$

One sum over the query terms. Each term contributes "how often it appears here, damped" times "how rare it is overall".

Usual settings: $k_1 \approx 1.5$ (range 1.2 to 2), $b \approx 0.75$ (range 0.5 to 0.8). There is no theory that fixes them; they are tuned.

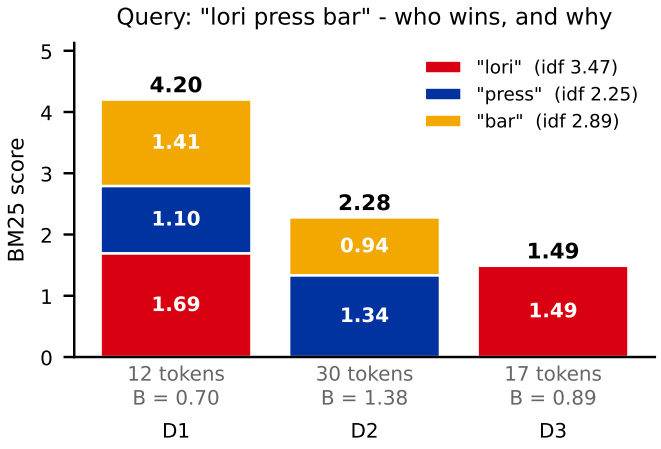
One chunk, scored by hand

Query `lori press bar`. D1: *"The Lori press operates at 2.5 bar during the first pressing stage."* - 12 tokens; the average chunk is 20. $k_1 = 1.5$, $b = 0.75$.

1. Length	$B = (1 - 0.75) + 0.75 \cdot \frac{12}{20}$	$= 0.70$	shorter than average, so $B < 1$
2. Saturation	$\frac{tf}{k_1 B + tf} = \frac{1}{1.5 \cdot 0.70 + 1} = \frac{1}{2.05}$	$= 0.4878$	each term appears once
3. Rarity	`lori`: 0.4878×3.472	$= 1.694$	
	`press`: 0.4878×2.253	$= 1.099$	
	`bar`: 0.4878×2.891	$= 1.410$	
	BM25($D1$)	$=$ **4.203**	

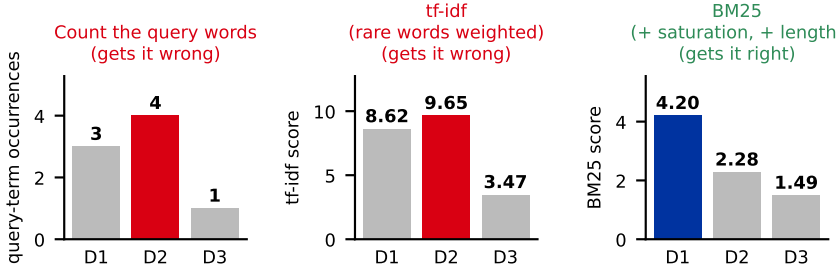
Every term gets the same damped count here, because each appears once in the same chunk. What separates them is idf: the rare word `lori` earns the most.

By hand, on three real chunks



D1 is short (12 tokens, $B = 0.70$) and hits all three terms, including the rare `lori`. **D2** says `press` three times but is 30 tokens long ($B = 1.38$) and never says `lori`. **D3** has only `lori`.

Three scorers, one right answer



D1 is the chunk that actually answers the question.

Counting is wrong. **tf-idf is also wrong.** Only once repetition saturates and length is discounted does D1 come first - and then it wins by nearly double. No larger model was involved at any point.

A formula can have decorative parts

You will often see BM25 written with an extra factor in the numerator:

$$\frac{tf \cdot (k_1 + 1)}{k_1 B + tf} \cdot \text{idf}(t) \qquad \text{instead of} \qquad \frac{tf}{k_1 B + tf} \cdot \text{idf}(t)$$

$(k_1 + 1)$ does not depend on the chunk or the term. It multiplies *every* score by the same constant.

It changes every number and **cannot change the ranking**. It exists for historical compatibility. Worth internalising in general: when you meet an unfamiliar formula, ask which parts can actually reorder the output.

Where this method is unbeatable

BM25 matches **literal strings**. That sounds primitive until you notice what people actually type:

```
PRS-400
Article 47.2
ISO 22000
invoice #88214
Paniryan
```

Part numbers, clause references, standards, invoice IDs, surnames. A model that "understands meaning" has no advantage here. There is no meaning. There is a string, and either you find it or you do not.

BM25 gives you a **guarantee**: if the term is in the chunk, that chunk gets a non-zero score. Keep the word guarantee in mind for two slides' time.

And where it stops working

Back to the pair from Section 2:

> **Chunk:** "Ripening cellars are held at 10 degrees and 85 percent humidity."
> **Query:** "How warm is the room where the cheese matures?"

Shared content words: **none**. `warm` $\neq$ `degrees`, `room` $\neq$ `cellars`, `matures` $\neq$ `ripening`.

Every term contributes $tf = 0$, so the sum is **exactly zero**. Not a low score. No score. BM25 does not think this chunk is a weak match, it does not see it at all.

The same literalism means `press`, `pressing` and `pressure` are three unrelated terms unless you bolt on a stemmer, and a stemmer for Armenian is its own project. This is called **vocabulary mismatch**, and it is why the next section exists.

Finding chunks by meaning

If matching words fails, match what the words mean.

You already have most of this

In LLM-2 you saw **word vectors**: in GPT-2's table, tokens used in similar contexts end up close together, with cosine similarity as the measure of close.

Two things change for retrieval:

The unit is a chunk

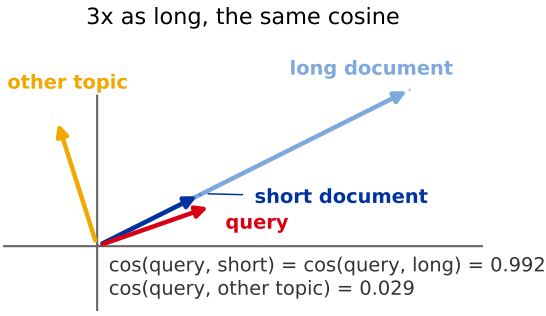
Not one word but a whole passage, compressed into a single vector.

The goal is retrieval

Not "similar in general" but specifically: *does this chunk answer this question?*

Those are not the same objective. A question and its answer are often not similar sentences at all. That difference is what the training has to fix.

Reminder: what cosine similarity measures



$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$$

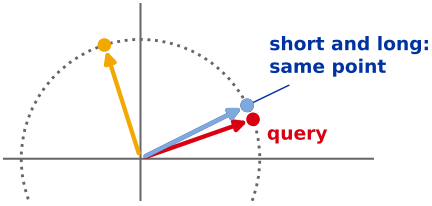
It measures the **angle** between two vectors, and deliberately ignores how long they are.

Here the long document is exactly 3 times the short one. Same direction, triple the magnitude.

Both score **0.992**. Straight-line distance would have called them very different: **0.14** versus **1.97**. For text that is the wrong answer, because a document should not become less relevant simply by being longer.

Which is why vectors get normalised

Normalise, and only the angle is left

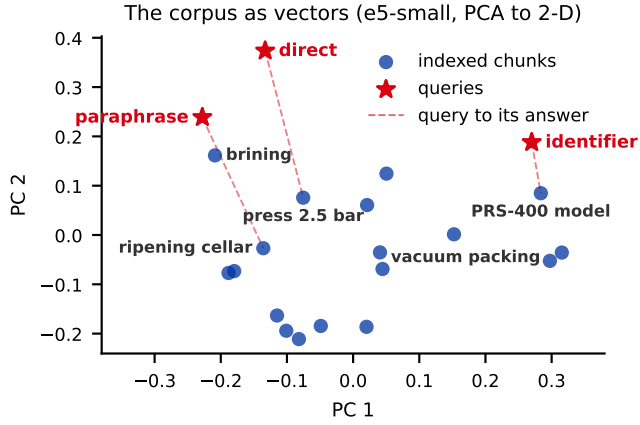


Divide every vector by its own length and they all land on the unit circle. The short and long documents collapse onto the **same point**, because they only ever differed in magnitude.

Once everything is normalised, the dot product *is* the cosine. So similarity search becomes one matrix multiplication, which is why it is fast.

This is why libraries have a `normalize_embeddings=True` flag, and why forgetting it quietly changes your rankings. Cosine of 1 means same direction, 0 means unrelated, and for real text embeddings (not toy arrows) you rarely see below about 0.6.

The corpus as points in space



Real vectors, real model. Retrieval is now: embed the question, take the nearest.

Careful: the multilingual model's vectors have **384** dimensions, so this 2-D shadow is an impression, not the retrieval signal. All **18** chunks are plotted, only a few labelled.

How the vectors are trained to be useful

Collect pairs that belong together: a question and the passage that answers it.

Pull each pair together. Push everything else apart.

$$\mathcal{L} \;=\; -\log \frac{\exp\big(\mathrm{sim}(q, d^{+})/\tau\big)}{\exp\big(\mathrm{sim}(q, d^{+})/\tau\big) \;+\; \sum_{j} \exp\big(\mathrm{sim}(q, d_{j}^{-})/\tau\big)}$$

Read it as a classification problem: among all candidate passages, the correct one d^{+} should get the highest similarity. τ is a **temperature**: small τ sharpens the competition and punishes near-misses harder.

This objective is called **InfoNCE**. The name is not important; the shape is: one positive on top, everything else pushed down.

The same loss, by hand

Query: `What pressure should the press run at for Lori cheese?`
Three candidate passages, real similarities from a real model:

	Passage	**sim**
d^+	the press runs at 2.5 bar	**0.834**
d_1^-	pasteurisation, 72 degrees	0.779
d_2^-	ripening cellar, 10 degrees	0.745

Divide each by τ, exponentiate, normalise so they sum to 1, then the loss is $-\log$ of the probability given to d^+:

	$p(d^+)$	$p(d_1^-)$	$p(d_2^-)$	**loss**
$\tau = 1.0$	0.350	0.331	0.320	1.051
$\tau = 0.1$	**0.504**	0.290	0.207	**0.686**
$\tau = 0.05$	**0.667**	0.221	0.113	**0.405**

What that table just told you about temperature

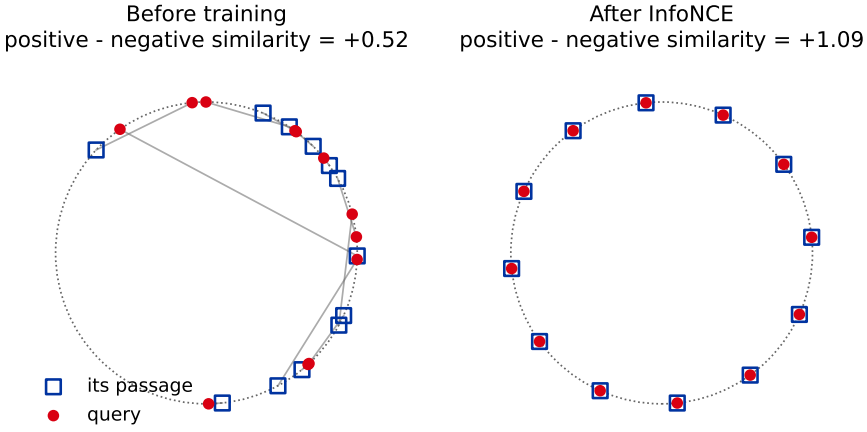
At $\tau = 1$ the three probabilities are **0.350, 0.331, 0.320**. The model has effectively said "any of these three, I do not mind".

And yet the similarities were 0.834 against 0.745 - the right answer really was ahead. The problem is that **all cosine similarities live in a narrow band**, so the gaps between them are small in absolute terms.

τ divides before the exponential - the same knob as LLM-6's sampling temperature - so a small τ **stretches small gaps into large ones**. It is the knob that turns "0.834 versus 0.745" into a confident decision, and it is why the loss drops from 1.05 to 0.41 without a single weight changing.

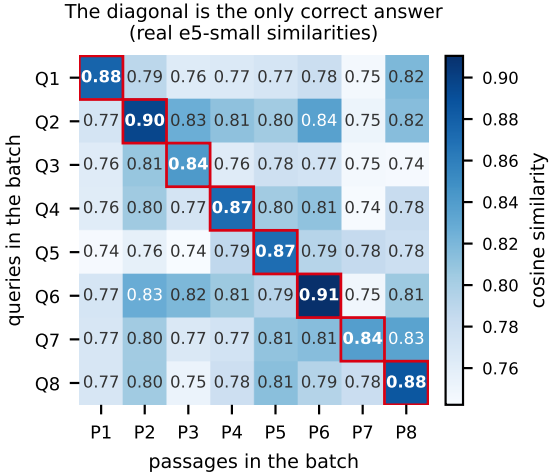
Keep this in mind: the narrow-band problem comes back twice more, once for chunking and once for Armenian.

What that objective actually does



A real training run on toy data. Each line joins a query to its own passage. Before training the pairs are scattered; afterwards each pair has collapsed onto a single point and the gap between "correct pair" and "any other pair" has roughly doubled.

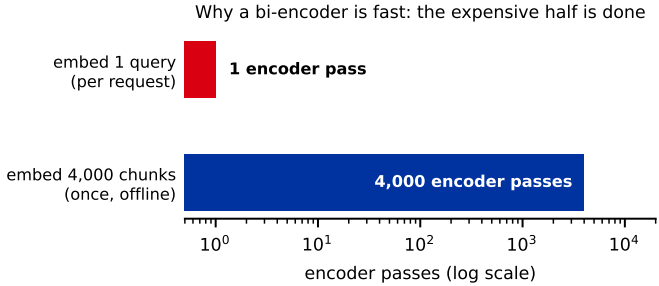
Where the negatives come from



For query i, the answer is passage i. Every other passage in the batch is a free negative.

64 pairs give 64 positives and 4,032 negatives, with no extra labelling. That is why contrastive training is cheap.

Why this can possibly be fast enough



The query and the chunk are encoded **separately**. So every chunk vector can be computed once, offline, and stored.

At query time the encoder runs **once**, on the question, then vectors are compared. That is Section 1's offline/online split, and the reason this is usable at all.

Back to the row we skipped

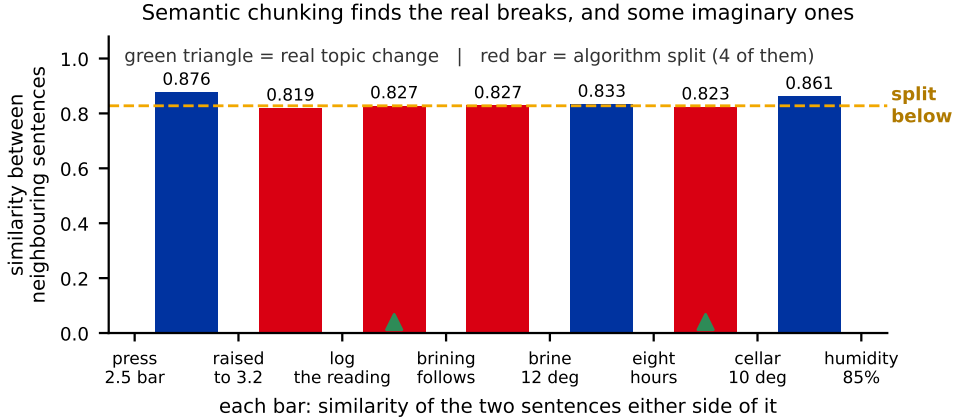
Remember the chunking table, and the one row we could not fill in:

Long undivided walls of text	**semantic**	nothing else has a boundary to use

You now have everything that row needs. Embed each sentence, measure how similar each one is to the next, and **cut where the similarity drops** - because a drop means the topic changed.

This is the same offline pass we just described, used for a different purpose: not to search the chunks, but to **decide where the chunks should begin**.

Semantic chunking, measured



On this passage the method found **both** real topic changes. It also invented **two more**.

Semantic chunking is not free

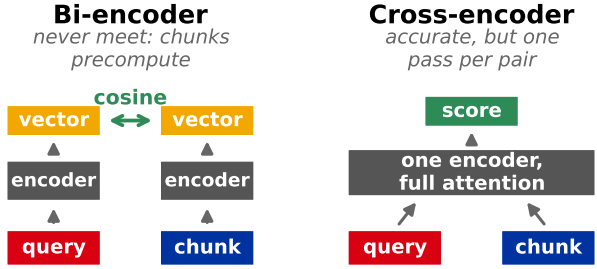
Look at why it over-split. Every similarity between neighbouring sentences sits between **0.819** and **0.876**:

0.876 0.819 0.827 0.827 0.833 0.823 0.861

The entire signal lives in a band **0.06 wide**. A threshold anywhere in there is making very fine distinctions on very little evidence.

Semantic chunking is a real technique, not a bad one. But it costs an embedding pass over your whole corpus and hands you a **threshold to tune**. Paragraph chunking costs nothing and has no knobs. Reach for this when you have measured that structure is failing you, not by default.

Two shapes: encode apart, or encode together



The **bi-encoder** is what we just built: fast, precomputable, and the query never looks at the chunk. The **cross-encoder** reads both together, is far more accurate, and must run once per pair - so it cannot search a million chunks.

Next lecture: search wide with the cheap one, re-score the survivors with the expensive one.

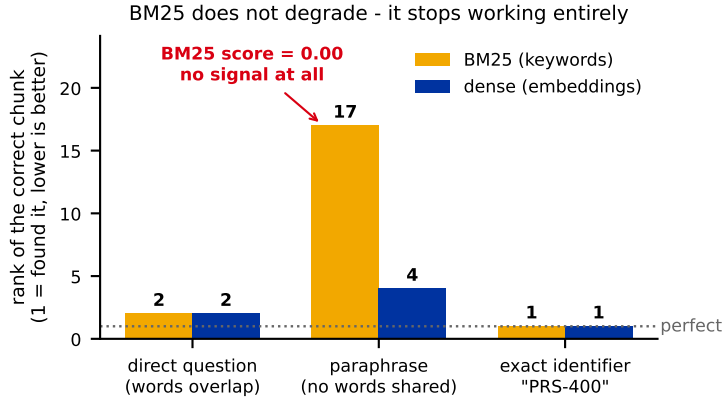
Practical: prefixes are not decoration

Many embedding models are trained **asymmetrically**: questions and passages are fed in with different prefixes.

At index time	`passage: Ripening cellars are held at 10 degrees...`
At query time	`query: How warm is the ripening room?`

Forget the prefixes and nothing crashes. You simply get quietly worse retrieval, with no error message anywhere. This is one of the most common silent bugs in a RAG system.

Now run the hard query through both



Measured on our 18-chunk corpus. On the paraphrase, BM25 scores exactly 0.000 and lands at rank 17 of 18; the embedding model puts the right chunk at rank 4.

The honest comparison

It is tempting to say "each method is better at different things". Our own measurements do not support that cleanly: on exact identifiers, the embedding model did *just as well* as BM25.

The real difference is not who scores higher. It is what each one **guarantees**:

BM25	**Embeddings**
Score is non-zero *if and only if* a query term literally appears. Fails **completely** on rephrasing.	Score is never exactly zero, and never guaranteed either. Degrades **gradually**, keeps finding roughly-related things.

One guarantees exact terms. The other covers rephrasing. Neither gives both - which is why the next lecture stops choosing.

When your language is not English

Everything so far assumed a good embedding model exists for your language.

The assumption hiding in Section 4

Every claim about embeddings assumed the model represents your language well. For English that is safe. For Armenian it is not.

Two compounding problems:

1. **Tokenization.** English-centric vocabularies fragment Armenian into many more pieces per word. You met this in the RNN chapter as the *Armenian tax*.
2. **Training data.** Multilingual models see far less Armenian than English, so Armenian occupies a smaller, blurrier region of the space.

Blurry means everything looks moderately similar to everything. A generic multilingual model scored our correct Armenian passage **0.854** and a completely unrelated one **0.797**. A gap of 0.057 is not much to build a threshold on.

Predict first

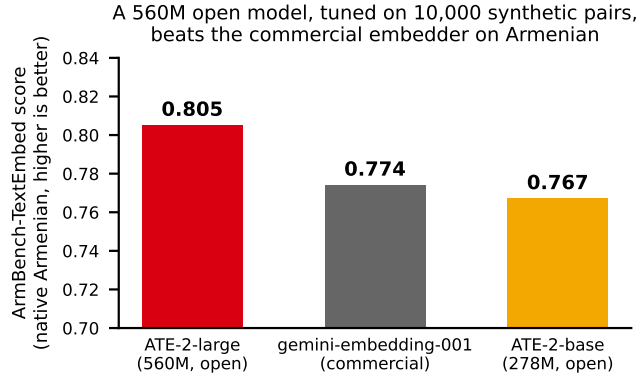
On an Armenian retrieval benchmark, which scores higher?

Google's commercial embedding model

A 560M open model fine-tuned on 10,000 synthetic pairs

Commit to an answer.

ATE-2



Armenian Text Embeddings 2 (ATE-2), from Metric AI Lab. Scores are the average on an Armenian embedding benchmark, native Armenian text, higher is better.

The recipe is smaller than you would guess



Take the multilingual **mE5** encoder. Build 10,000 training pairs by machine-translating English Reddit title/body pairs into Armenian. Fine-tune. That is the whole method.

The pairs are **noisy** translations, not a hand-curated corpus, and the reported gain over the starting model is around 11 to 12 percent on average.

Two lessons worth more than the numbers

1. **Bigger and more general is not automatically better for your language.**

   A 560M open model beat a large commercial one, because it was pointed at one language instead of a hundred.

2. **Targeting beats volume.**

   Ten thousand noisy, automatically generated pairs were enough to match what far larger training sets buy. You do not need a million examples to make a real difference in a low-resource language.

If you build Armenian retrieval, start from a model adapted to Armenian, and remember that producing your own domain pairs is a realistic project, not a research programme.

Recap

- **RAG** is: retrieve the right text, then let the model read it. The retrieval is the hard part.
- It is **two pipelines**. Indexing runs once, offline; only the query path runs per question.
- **Chunking sets the ceiling** - nothing downstream recovers what it loses. Paragraph chunking is the cheapest good default.
- **tf-idf** = frequent here, rare elsewhere. Most of the way there, and it still ranked our example wrong.
- **BM25** = tf-idf plus two repairs: repetition saturates, long chunks get discounted. Unbeatable on exact strings, *exactly zero* on rephrasing.
- **Cosine** measures angle, not length, so a long document is not penalised for being long.
- **Embeddings** are trained contrastively to pull questions and answers together, and degrade gracefully where BM25 stops dead.
- For **Armenian**, a targeted fine-tune on 10,000 pairs beat a commercial model.

You can now retrieve.

You have two methods with complementary guarantees,
and no principled reason to pick one.

Next: extract the keywords, fuse the two ranked lists, and re-score what survives with a model that finally lets the query and the chunk look at each other.

Whether combining them actually helps is a **measurement**, not a given. Hold on to one fact from today: a BM25 rank means nothing when the score is zero.